

# Recovering the Hidden Digit, Part II

## Diffusion-prior posterior sampling for ERT

Travis Whitney   Cole Seifert   Alexander Nutt

AI 539, Oregon State University

Project 2 Competition, Spring 2026

## Same problem, new rule

---

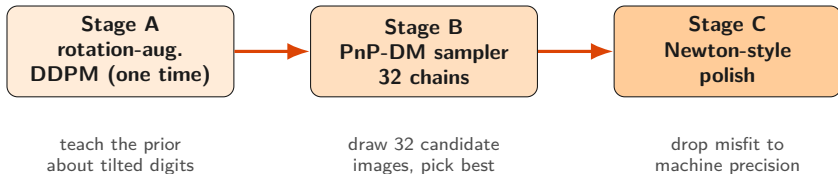
The physics from Project 1 hasn't changed. We still have a hidden  $28 \times 28$  digit, a list of 1900 boundary voltage readings, and the forward simulator  $F$ . The job is still to work backward from the readings to the digit.

What changed is the rule on the prior. In Project 1 we used the 60,000 MNIST training images as a lookup table. For Project 2 the prior has to be a diffusion model. A diffusion model is a neural network that learned what a digit looks like by practicing on the MNIST set. We can ask it to draw a fresh digit, but it does not know about our voltage readings.

The rest of this talk is how we combined a diffusion model with the ERT measurements. The final misfit is  $6.16 \times 10^{-13}$ , recovering a tilted 5.

## The pipeline

---



Stage A is a one-time setup. After that, every run goes through Stage B and Stage C. Total wall time on a MacBook is about eight minutes.

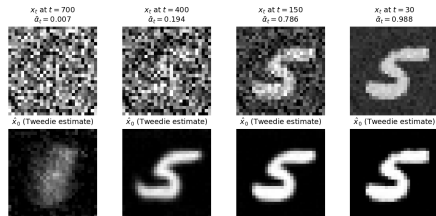
# What a diffusion model lets us do

A diffusion model is trained to look at a noisy image and predict the noise that was added. Once it can do that, simple algebra gives us a guess at the clean image underneath:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

So at any moment along a noisy chain, the network gives us its best guess at the clean digit underneath. At high noise the guess is fuzzy. At low noise it is sharp. Every method here leans on that one identity.

Tweedie's formula at four noise levels (top: input  $x_t$ ; bottom: network's clean-image guess  $\hat{x}_0$ )



Top: a noisy input at four noise levels. Bottom: the network's clean-image guess at each.

## What we tried first

---

**The idea.** Train a diffusion model on the 60,000 MNIST digits so it knows what handwritten digits look like. Then, instead of asking it to draw random digits, steer its drawing process at every step using the ERT measurements. The result should be a digit that looks realistic and matches the data.

The steering works by asking, at every step, “if this were the final image, how far off would the predicted voltages be from what we measured?” Then nudge the image to reduce that mismatch.

This is the standard recipe in the literature. It is called DPS, short for Diffusion Posterior Sampling.

**What happened.** We ran this five separate times from different random starting noise.

**Four of the five runs agreed the digit is a 4.**

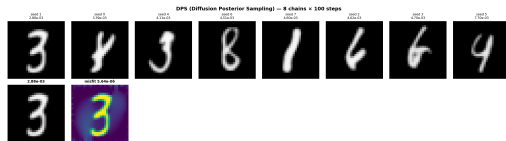
Project 1 already showed the digit is a 5. So this approach gave us the wrong answer, consistently.

## Why it got the wrong digit

Two things broke at once.

- ▶ The little nudge at each step is in the direction of the data gradient. That gradient is dominated by directions the boundary sensors hardly see, so most of the nudge goes to waste.
- ▶ The drawing process only has about 100 steps. A hundred weak nudges are not enough to override the prior's pull toward the most common-looking digit.

The chain commits early to whatever upright digit shape is closest to the data. For this measurement vector, that was a 4.



Eight chains from our DPS run, each locked onto a wrong digit (3, 4, 6, 8). No 5 in sight.

## The fix: split the work between two variables

---

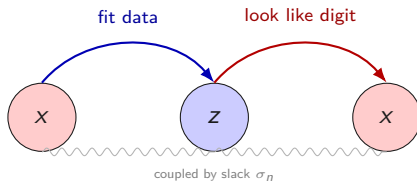
DPS asks one variable to do two jobs at once: look like a digit, and fit the data. The two pulls compromise on every step, and the smaller one loses.

Instead, introduce a second variable. Call them  $x$  and  $z$ . Give them separate jobs:

- ▶  $x$  only has to look like a digit.
- ▶  $z$  only has to fit the measurements.
- ▶ A soft penalty keeps  $z$  near  $x$ .

Then alternate. Update  $z$  to fit the data. Update  $x$  to look like a digit. Repeat. Each update does one thing well.

This sampler is called PnP-DM (Wu et al., NeurIPS 2024). We run 80 alternations per chain, 32 chains in parallel.



## What the data step actually minimizes

---

The data step picks the  $z$  that does two things at once: fits the measurements, and stays close to the current  $x$ . Written as an objective:

$$\mathcal{L}(z) = \underbrace{\frac{1}{2\sigma_{\text{meas}}^2} \|y_{\text{obs}} - F(\sigma_{\text{bg}} + z)\|^2}_{\text{data misfit}} + \underbrace{\frac{1}{2\sigma_n^2} \|z - x\|^2}_{\text{coupling penalty}}.$$

### Piece by piece.

- ▶  $y_{\text{obs}}$  is the 1900 measured voltages.  $F(\sigma_{\text{bg}} + z)$  is what the simulator would predict for our candidate. The squared norm is the sum of per-electrode errors.
- ▶  $\sigma_{\text{meas}}$  is how noisy we think the measurements are. Smaller  $\sigma_{\text{meas}}$  means we trust them more, so the first term gets a bigger weight.
- ▶  $\|z - x\|^2$  pulls  $z$  back toward the current  $x$ . The weight  $1/\sigma_n^2$  depends on the slack: tight slack means strong pull.

**Reading it.** “Find a  $z$  that explains the measurements, but do not stray too far from the current digit guess.” At the start of the chain  $\sigma_n$  is big (loose leash), so the data wins. As the schedule anneals,  $\sigma_n$  shrinks and the coupling wins, forcing  $z$  and  $x$  to agree.

## How we solve it: one Gauss-Newton step

---

DPS takes a small step in the gradient direction. We instead linearize  $F$  around the current  $z$ , plug the linearization into  $\mathcal{L}$ , and minimize the resulting quadratic. The minimum is the solution of a small linear system in the update  $\Delta$ :

$$\underbrace{\left( \frac{1}{\sigma_{\text{meas}}^2} J^\top J + \frac{1}{\sigma_n^2} I \right)}_{\text{curvature, with damping}} \Delta = \underbrace{-\frac{1}{\sigma_{\text{meas}}^2} J^\top r}_{\text{data pull}} + \underbrace{\frac{1}{\sigma_n^2} (x - z)}_{\text{coupling pull}}.$$

**Piece by piece.**

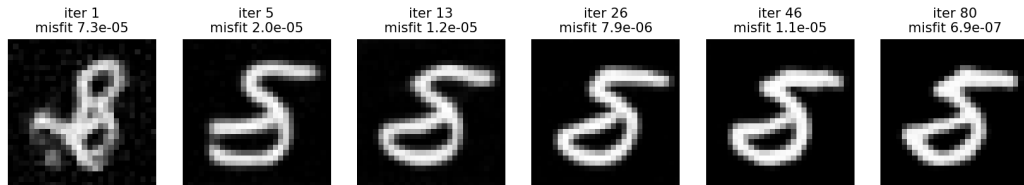
- ▶  $\Delta$  is the update we are solving for. After solving:  $z \leftarrow z + \Delta$ .
- ▶  $J^\top J$  is the curvature of the data term. In directions where  $J$  has big columns (sensitive pixels), the curvature is big and the step is small. The  $\sigma_n^{-2} I$  damping fills in the directions  $J$  cannot see, so the matrix is always invertible.
- ▶  $r = F(\sigma_{\text{bg}} + z) - y_{\text{obs}}$  is the current residual. Multiplying by  $J^\top$  back-projects it onto pixels: “which pixels would reduce this residual.”
- ▶  $(x - z)$  is the coupling restoring force: pulls  $z$  back toward the current  $x$ .

One Gauss-Newton step does the work of dozens of gradient steps on ill-conditioned  $J$ .

## One chain, six frames: noise becomes a 5

---

**One PnP-DM chain over 80 iterations: pure noise to a clean tilted 5**

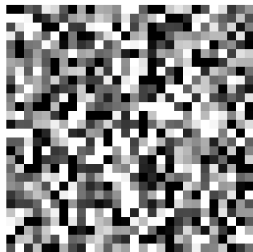


**Reading left to right.** At iteration 1 the image is essentially random noise. By iteration 5 a digit shape is forming. By iteration 26 the digit identity (a 5) is settled, though the strokes are still a bit smudged. By iteration 80 the chain has converged to a clean tilted 5 with misfit around  $10^{-7}$ . Every other chain looks similar.

## Same chain, animated: watch the digit class lock in

---

PnP-DM on competition vector (digit 5)



iter 0 / 80

misfit 3.75e-02

The filmstrip on the previous slide showed six time-points. Here is the same chain animated through all 80 iterations.

**What to watch for.** In the first  $\sim 5$  iterations the image is still essentially noise. Around iteration 10 to 15 the **data step** (Gauss-Newton) has accumulated enough information about the voltages that the iterate starts to look like a coherent shape. By iteration 20 the digit class is locked in: this chain will be a 5, not an 8 or a 3. From there on, the prior step polishes the strokes and the data step nudges the pixels for a tighter fit.

The digit identity is decided early. The rest is refinement.

## Where each chain starts, and why answers can disagree

---

**Where does a chain start?** Not from the measurements. Each chain starts from a fresh draw of pure Gaussian noise:  $x_0 \sim \mathcal{N}(0, I)$ . That is just a  $28 \times 28$  grid of random numbers (TV static). The measurements  $y_{\text{obs}}$  enter only through the data step, which runs at every iteration. The data step pulls toward fitting  $y_{\text{obs}}$ . The prior step pulls toward looking like a digit. Together they convert the random noise into the recovered digit.

**Why several different chains can give different digits.** The forward map is a  $784 \rightarrow 1900$  blur. Two different digits can produce voltage readings that agree to many decimal places. So the question “which digit is inside?” does not have a unique answer from the readings alone. The posterior has *several* valleys, one per plausible digit.

A single chain commits to whichever valley its random starting noise rolls it into. Different chains, different starting noises, different valleys. Across our 32 chains, most land on the 5 valley (the correct one for this  $y_{\text{obs}}$ ), a few land on the 8 valley. We pick the chain with the lowest data misfit. The 5 wins every time.

**The 32 chains are a safety net. The lowest-misfit chain wins the vote.**

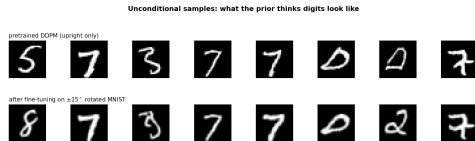
This is also why one held-out test (digit 3) recovers as a 5. That particular 3 and a particular 5 produce nearly identical voltage readings, so no method can tell them apart from the data alone.

## One more piece: rotation augmentation

The pretrained MNIST diffusion model has only seen upright digits during training. Our truth is a 5 tilted by about 7.5 degrees. During PnP-DM the prior step keeps nudging back toward upright, while the data step pulls toward tilted. Without intervention the prior wins.

The fix is to keep training the diffusion model a little longer, but on rotated copies of the MNIST digits. Five epochs of this on a MacBook takes about twenty minutes.

After the extra training, the model thinks tilted digits are just as normal as upright ones. The prior step stops fighting the data, and the chain converges to the truth.



Top row: random digits from the pretrained model.

Bottom row: random digits from the same model after the extra rotation training.

## Finishing the job: a Newton-style polish

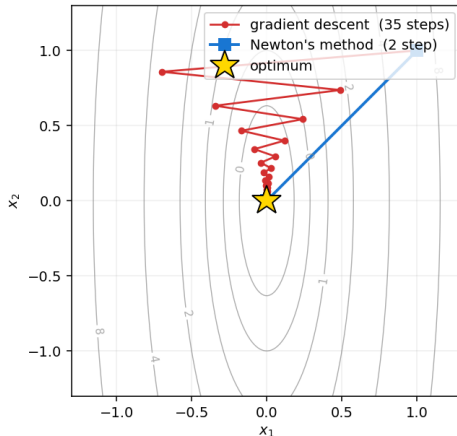
After PnP-DM the best chain has misfit about  $1.8 \times 10^{-7}$ . We know the digit is right. The rest is a local optimization on the pixels.

Project 1's polish was gradient descent. It converges linearly, and the steps get tinier as we approach the optimum.

Instead we use a Newton-style step, which accounts for the curvature of the data-fit landscape. Each step roughly squares the error, so a handful of steps drops the misfit many orders of magnitude. We hit the float64 floor in about 25 seconds.

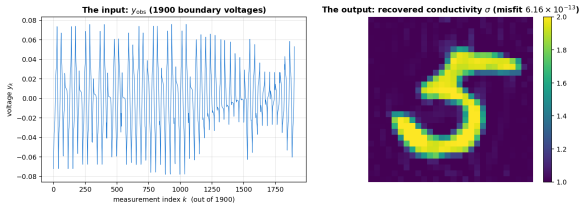
**Final misfit:  $6.16 \times 10^{-13}$ .**

**Elongated quadratic bowl: gradient descent zigzags; Newton's method lands at the bottom in one shot**



GD zigzags; Newton jumps to the bottom.

# What we recovered

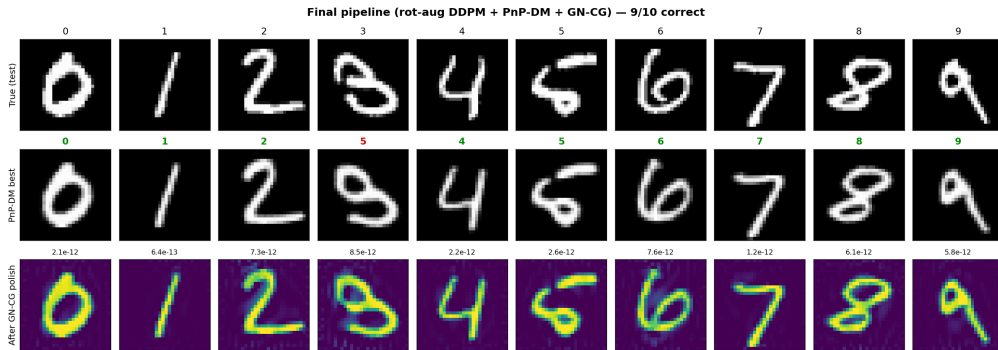


The recovered digit is a **5**, tilted by about 7.5 degrees. The final misfit is  $6.16 \times 10^{-13}$ .

That is roughly 163,000 times tighter than Project 1's rotation-aware result on the same data. The pipeline ran all the way down to the limit of double-precision arithmetic.

Most of that big multiplier comes from the polish, not from the diffusion. The diffusion did its job by putting us in the right digit basin. Once there, the Newton-style polish beat Project 1's gradient polish by about five orders of magnitude.

## Same method on 10 unseen digits: 9 out of 10



Labels read **true to recovered**. For each test digit we synthesized fresh measurements, ran the full pipeline, and asked a classifier to read the recovered image. The one miss (3 read as 5) is a genuine ambiguity of the physics: that particular handwritten 3 has a curled bottom that produces voltage readings extremely close to those of certain 5s. The other nine cases polish to the same  $10^{-12}$  regime as the competition vector.

## Three of those chains, animated, in parallel

---

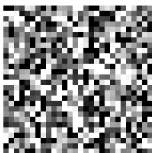
PnP-DM on a held-out 1



iter 0 / 80  
misfit 8.49e-02

held-out 1

PnP-DM on a held-out 7



iter 0 / 80  
misfit 6.38e-02

held-out 7

PnP-DM on a held-out 8



iter 0 / 80  
misfit 4.15e-02

held-out 8

Three different held-out test images, three independent chains, all running with the same hyperparameters and the same diffusion model. Each chain starts from its own fresh Gaussian noise. Notice the common pattern: in the first  $\sim 15$  iterations the chain commits to a digit class, and the remaining  $\sim 65$  iterations sharpen the strokes and drive the misfit down. There is no template database anywhere in the pipeline. The diffusion prior and the Gauss-Newton data step do all the work.

## Summary

---

- ▶ Project 2 forced us off template matching and onto a diffusion prior. The first thing we tried, DPS, gave the wrong digit every time because its data step is too weak.
- ▶ Variable splitting separates the two jobs. PnP-DM replaces the gradient nudge with a Gauss-Newton solve, which commits the chain to the right digit in a few iterations. 32 chains in parallel cover the posterior modes.
- ▶ Rotation augmentation is a one-time twenty-minute fix that unlocks the diffusion prior for tilted digits.
- ▶ A Newton-style polish replaces gradient descent at the endgame and drops the misfit by about five extra orders of magnitude.
- ▶ **Result:** digit 5, final misfit  $6.16 \times 10^{-13}$ , held-out 9 out of 10.

**Thank you. Questions?**